PyTorch Bootcamp

Fundamentals of Artificial Intelligence, Machine and Deep Learning

A. A. 2025/26

Slide credits: *Chiara Plizzari*

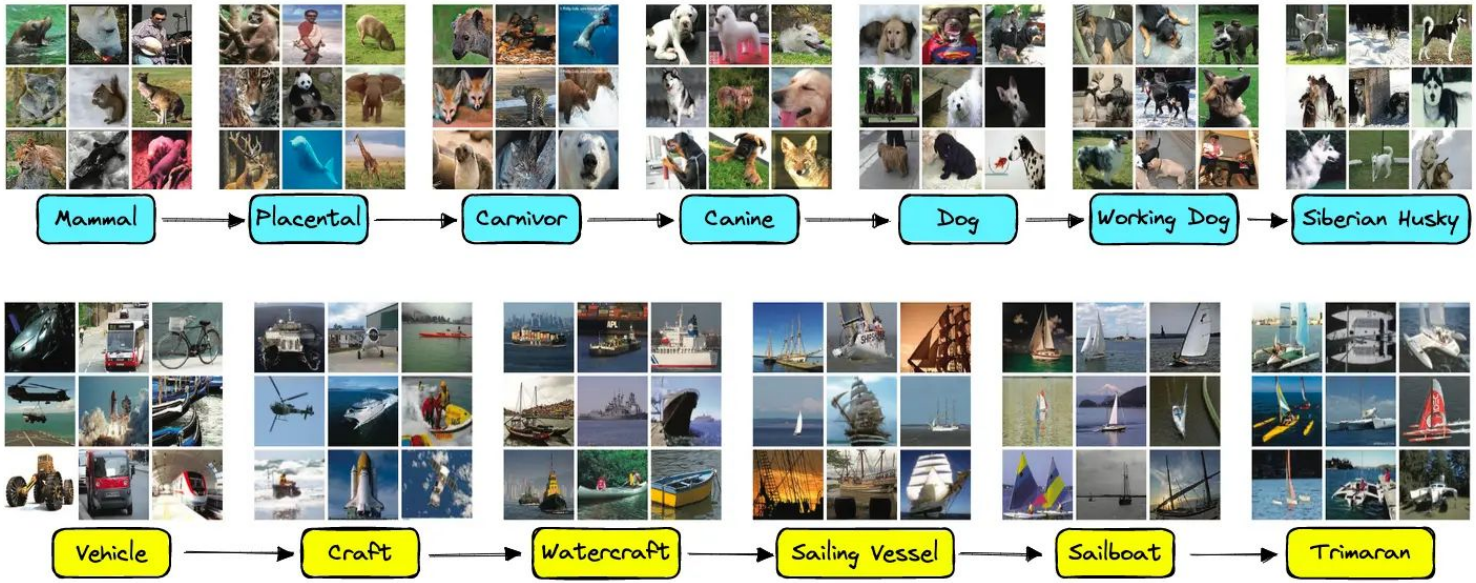
Teaching Assistants: *Davide Sferrazza, Stephany Ortuño Chanelo*

Training CNNs: AlexNet & ResNet

Lesson 4

ImageNet

Released in 2009, the dataset comprised 12 million images across 22,000 categories.
ImageNet Large Scale Visual Recognition Challenge (ILSVRC).



AlexNet

AlexNet was the first model to score <25% error rate.

A Neural Network introduced in the paper: Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems* (2012).

- 227x227 inputs
- 5 Convolutional layers
- Max pooling
- 3 fully-connected layers
- ReLU non-linearities

AlexNet

AlexNet produced *a **lot** of parameters*, making training either impractically slow or simply impossible (at that time). AlexNet was distributed across two GPUs.

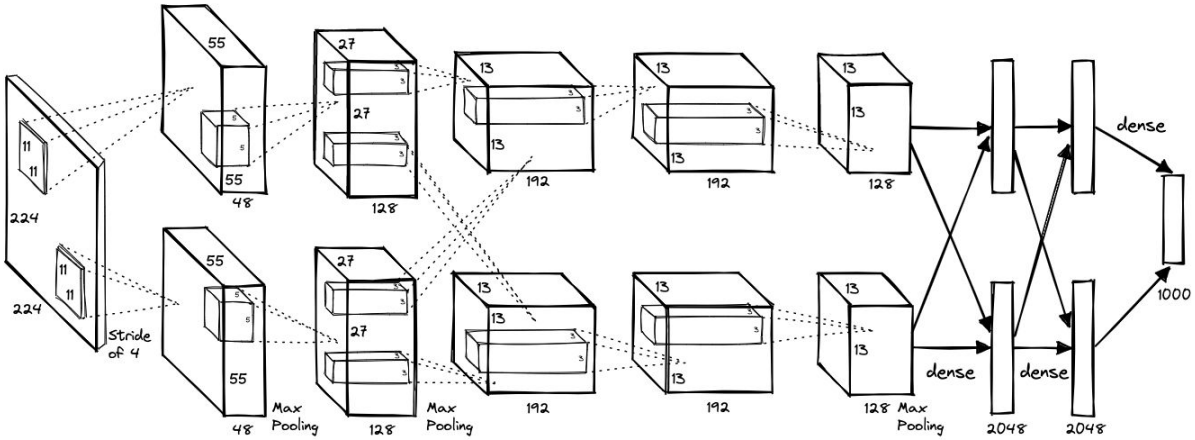


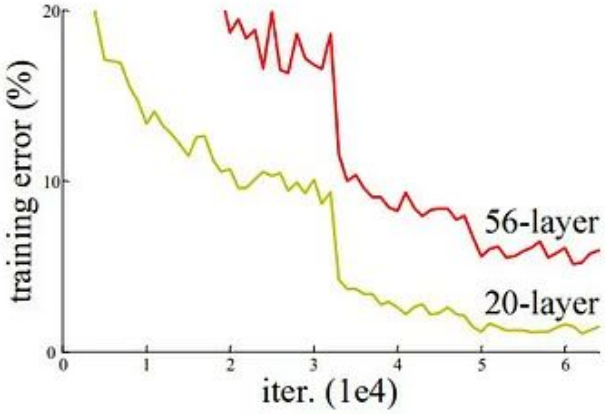
Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

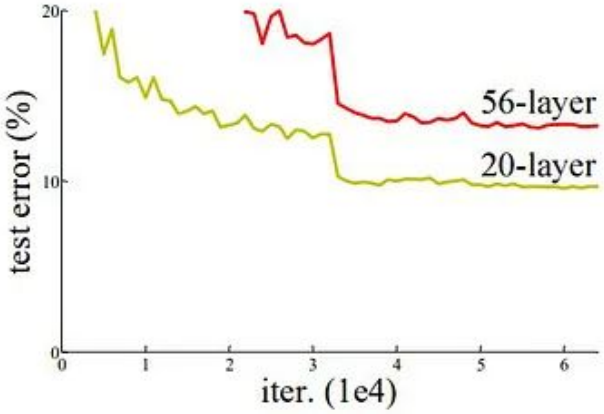
AlexNet

	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0

Consecutive Models

Every consecutive winning architecture uses **more layers** to lower the error rate.





The training and test error rate similarly increases as the number of layers is increased.

Issue: Performance Degradation → Vanishing/Exploding gradient.

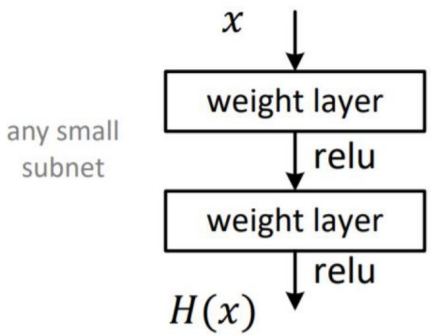
Residual Connections

Each layer adds something to the previous value, rather than producing an entirely new value.

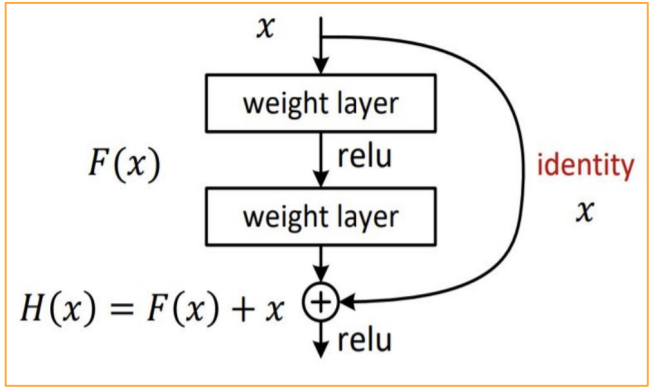
Advantages:

- Allow **gradients to flow** through identity **shortcuts.**
- Simplifies training by learning **residual functions** instead of full mappings.

Plain

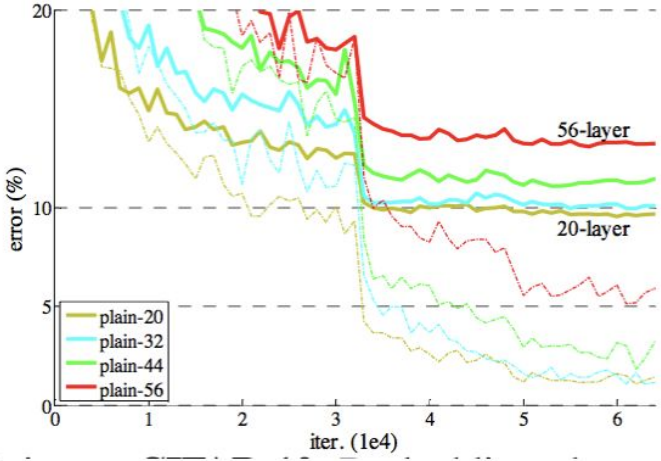


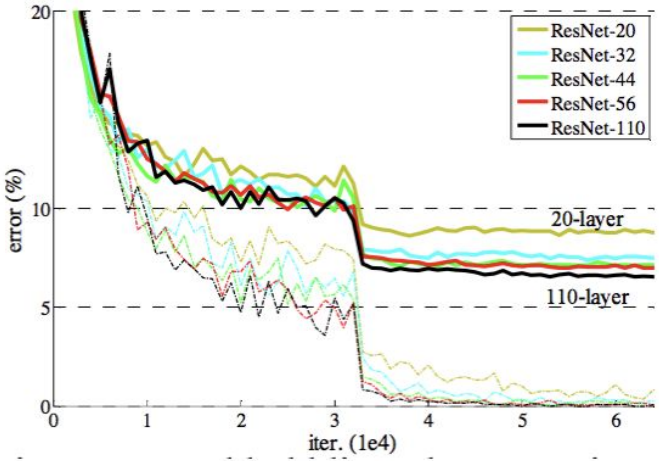
Residual



Deep Residual Network

- Deep Residual Networks (ResNets) consist of many layers of residual units.
- Performance on CIFAR-10, a small object recognition dataset:





- For a regular convnet, performance declines with depth, but for a ResNet, it keeps improving

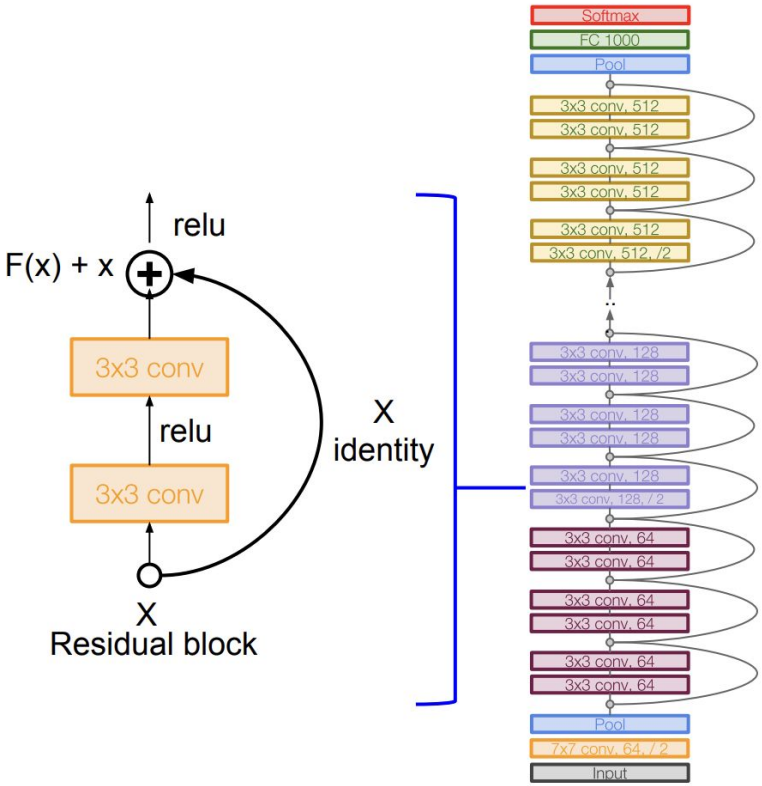
Easier and Faster Optimization

- **Hypothesis:** the problem is an optimization problem, deeper models are harder to optimize.
- The deeper model should be able to perform at least as well as the shallower model.
- A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.

ResNet

Full ResNet architecture:

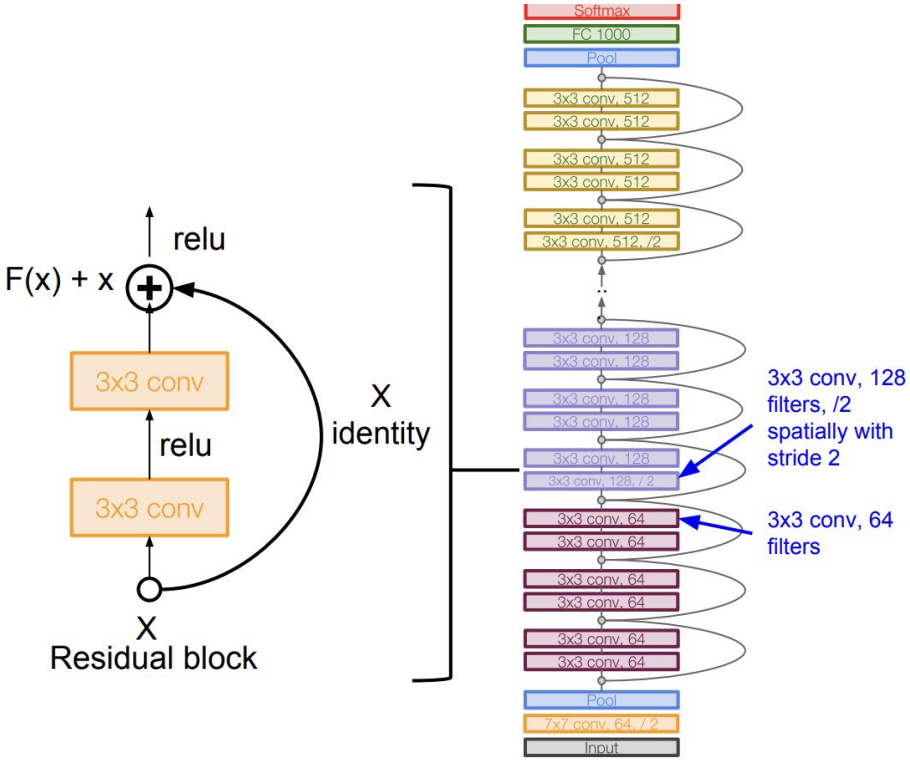
- Stack residual blocks
- Every residual block has two 3x3 conv layers



ResNet

Full ResNet architecture:

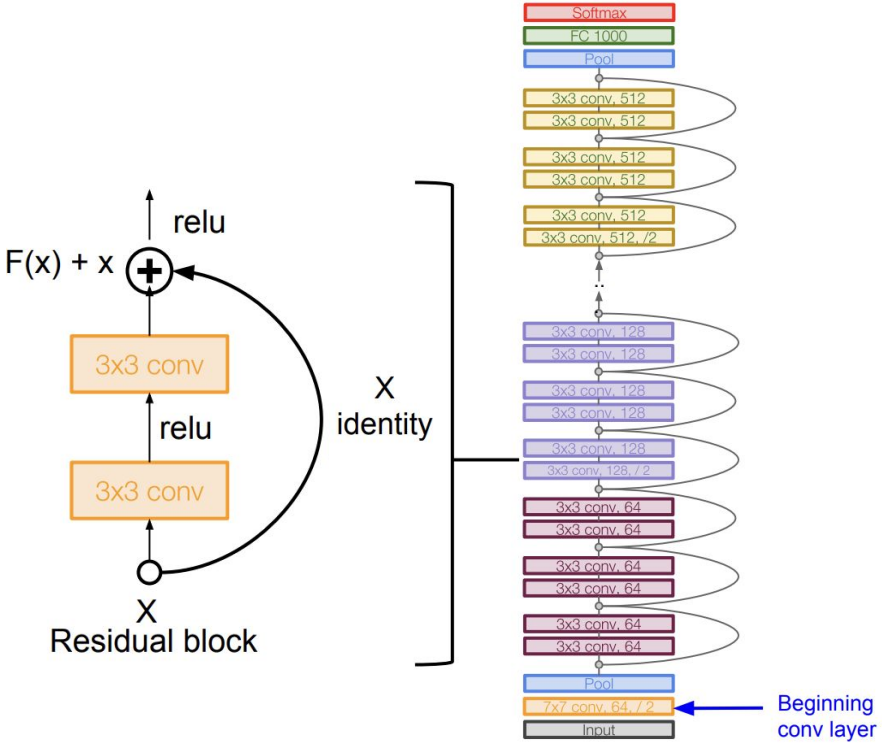
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)



ResNet

Full ResNet architecture:

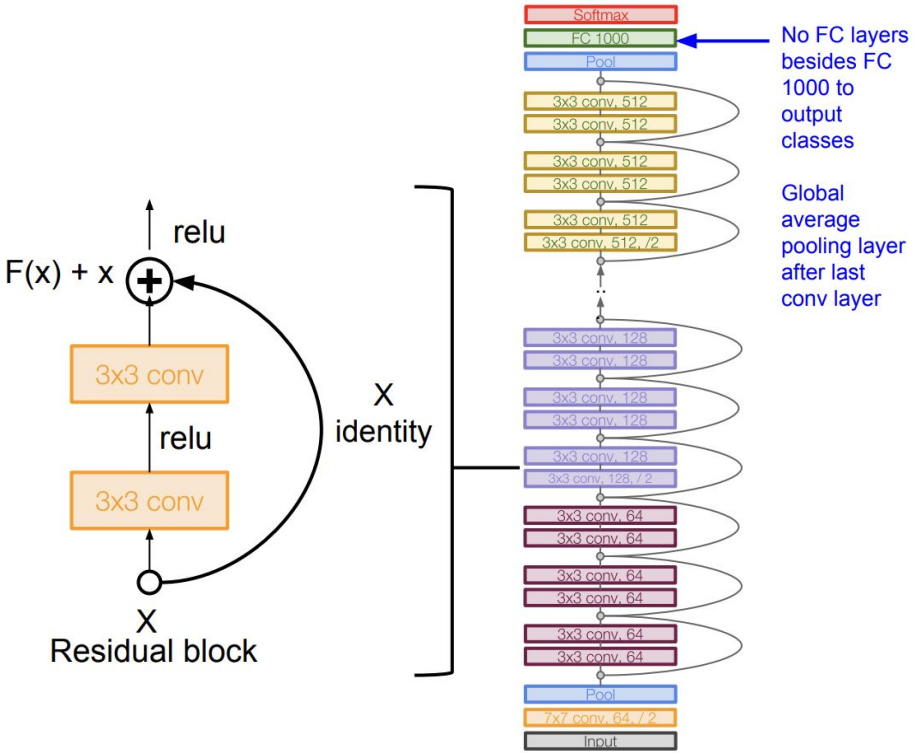
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning



ResNet

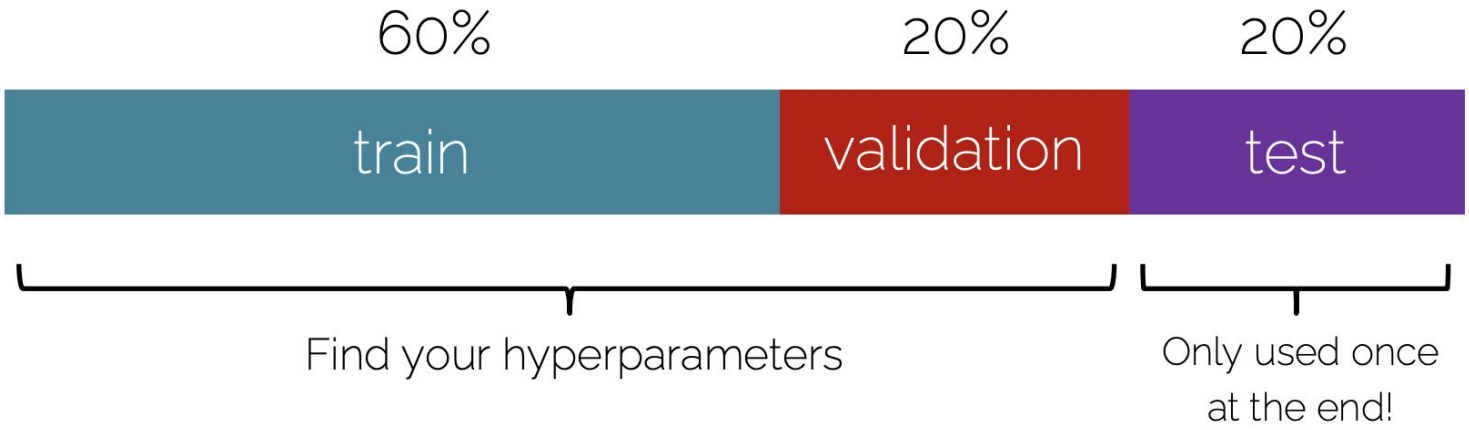
Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning
- No FC layers at the end (only FC 1000 to output classes)



How to find good hyperparameters?

- Split your data!



How to find good hyperparameters?

- Network architecture (e.g., num layers, hidden layer, activation function)
- Number of iterations (i.e., epochs)
- Learning rate(s)
- Regularization (i.e., decay, etc.)
- Batch size

Your turn!

- At this link http://cs231n.stanford.edu/tiny-imagenet-200.zip you can find the TinyImageNet dataset.
- Train an AlexNet network on it, by performing a hyperparameter search.
- Report results for:
  - LR: 0.1, 0.001, 0.0001
  - Batch Size: 16, 32, 64
- What is the effect of changing the decay to the optimizer?
- Do the same analysis for the ResNet architecture.

Notebook @
https://colab.research.google.com/drive/1WewopJh14L9zvdxnon1LRK2ruixFPzTA#scrollTo=mgA4k7F9OWxM